CMPE180A — Dr Gomez — Spring 2026
Sample Exam 2 — Explicit Version

Practice instructions: These sample questions are written in a more explicit style. Read each prompt carefully, pay attention to the requested function behavior, and use the sample input/output to guide your interpretation.

1. Q1. Merge Two Sorted Linked Lists

Write a function Node* mergeSortedLists(Node* a, Node* b) that merges two sorted singly linked lists into one sorted list.

Do not create a new list of values; reuse the existing nodes by changing pointers.

Sample input A: 1 -> 4 -> 8

Sample input B: 2 -> 3 -> 10

Expected output: 1 -> 2 -> 3 -> 4 -> 8 -> 10

State the time complexity.

2. Q2. Recursive Palindrome Checker

Write a recursive function bool isPalindrome(const string& s, int left, int right) that checks whether a string is a palindrome.

Your function should compare characters at symmetric positions and recurse inward.

Sample input 1: "racecar"

Expected output 1: true

Sample input 2: "recursion"

Expected output 2: false

State the base case(s) clearly.

3. Q3. Queue Using Linked List Nodes

Design a queue class implemented with linked list nodes.

Your class must support enqueue(x), dequeue(), front(), and isEmpty().

Show what the front and rear pointers should do after each operation.

Sample operations: enqueue(5), enqueue(9), dequeue(), enqueue(2), front()

Expected dequeue result: 5

Expected front result after the sequence: 9

Explain why enqueue and dequeue are O(1).

4. Q4. Backtracking Maze Path Finder

Given a 2D grid where 0 means open cell and 1 means blocked cell, write a backtracking algorithm that determines whether a path exists from the top-left corner to the bottom-right corner.

You may move only right or down.

Sample input grid:

[[0, 0, 1],

[1, 0, 0],

[1, 1, 0]]

Expected output: path exists

Briefly explain how your algorithm avoids revisiting invalid choices.